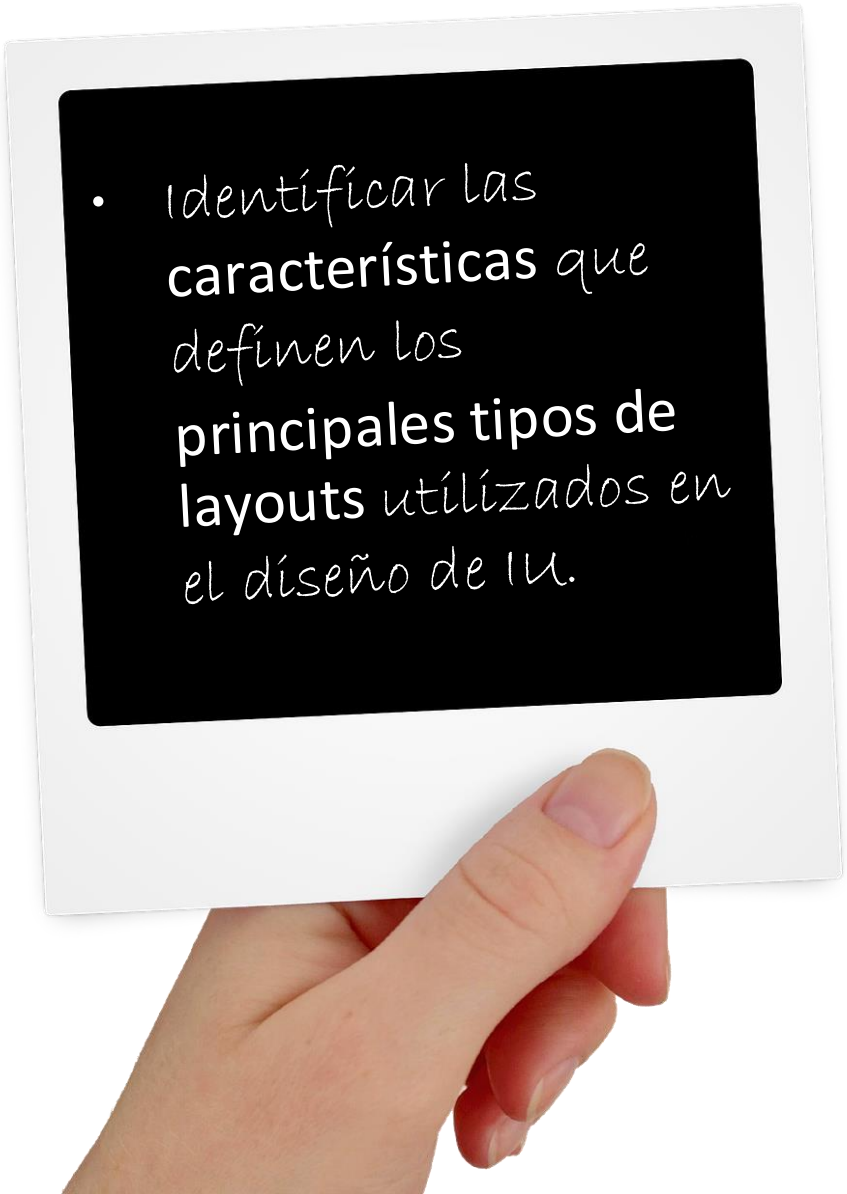


TIPOS DE LAYOUTS

Liliana Patricia Santacruz Valencia

Universidad
Rey Juan Carlos

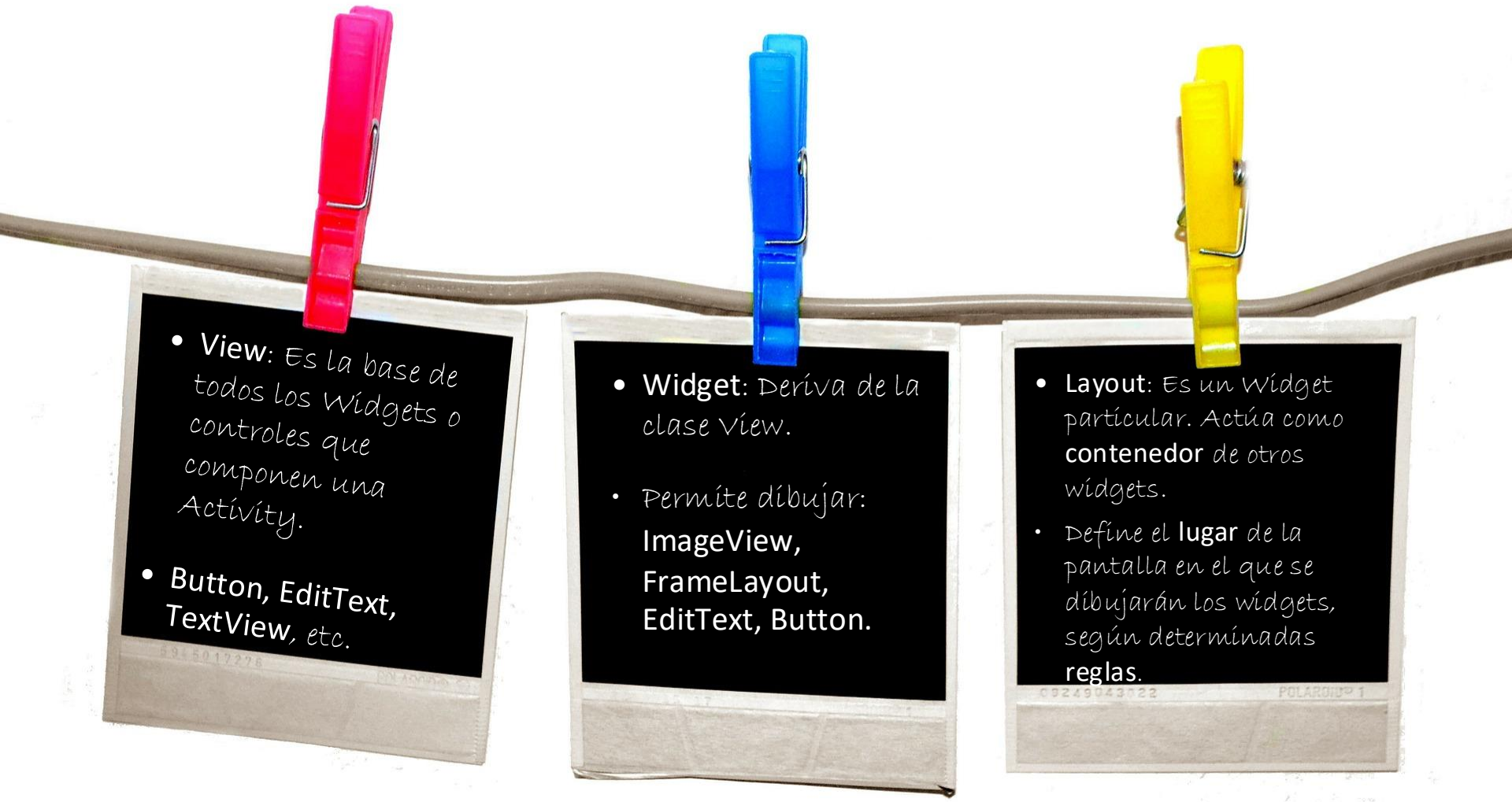
Objetivo


• Identificar las características que definen los principales tipos de layouts utilizados en el diseño de IU.

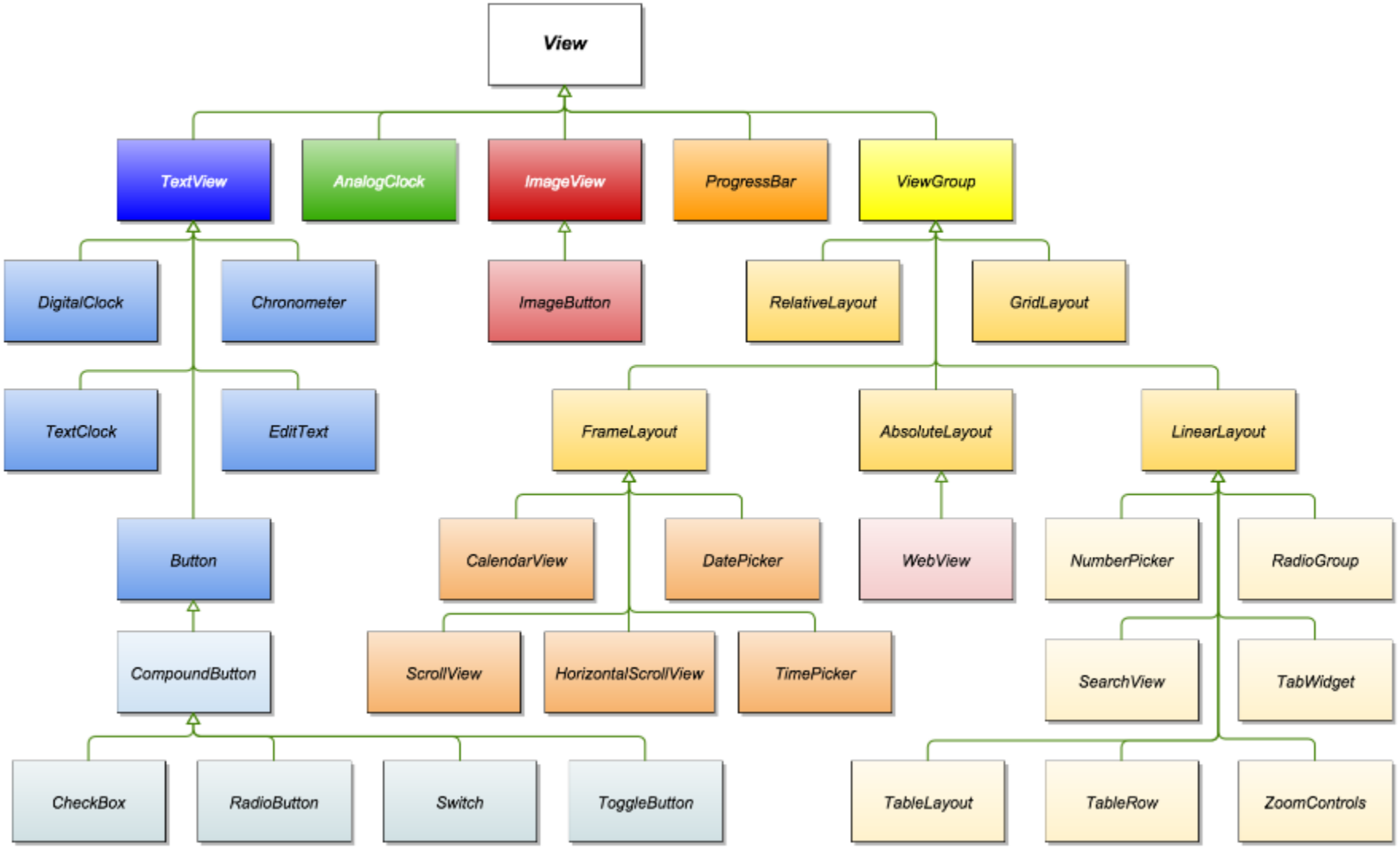
Tipos de Layouts



Definición

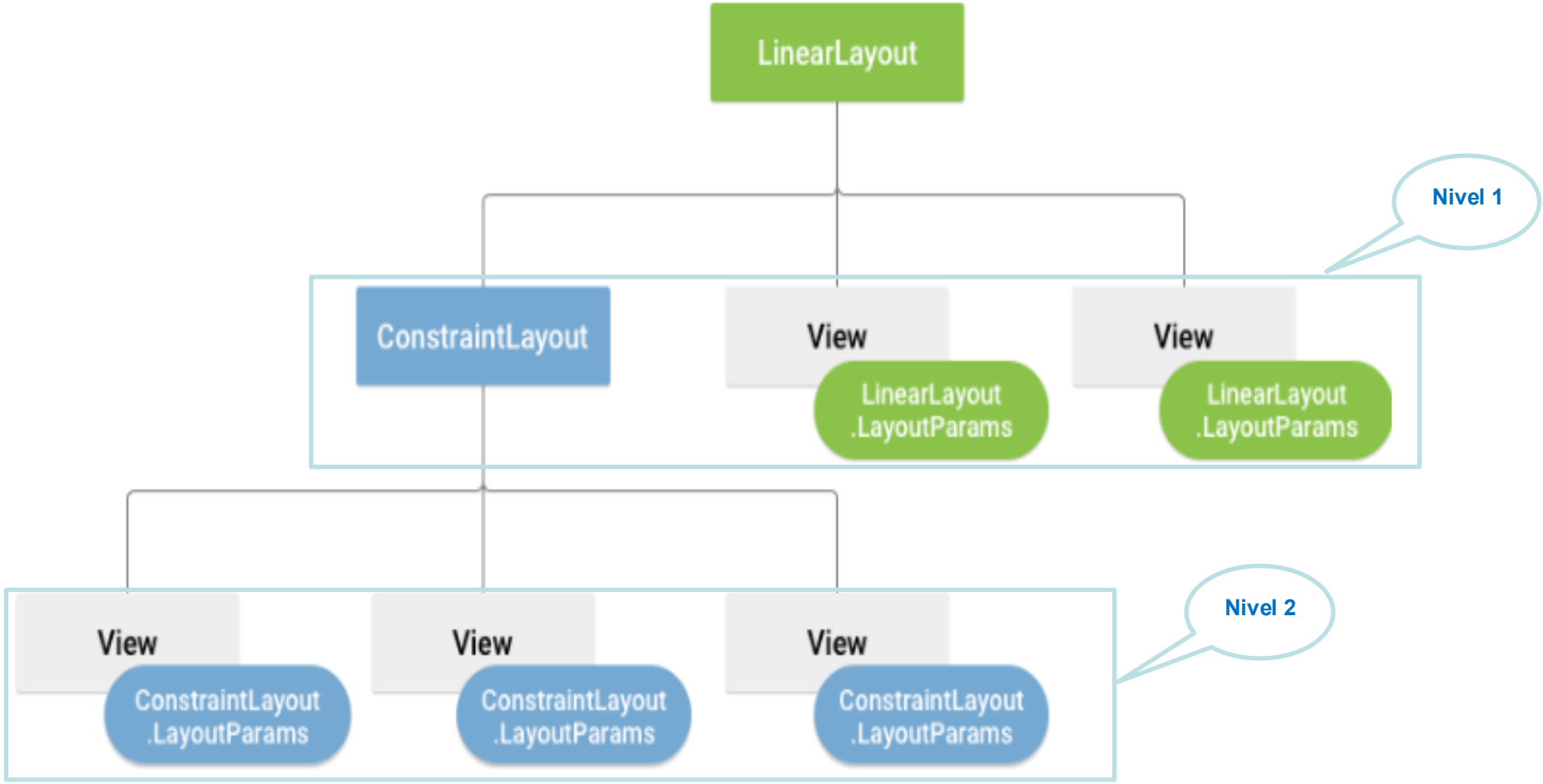

• View: Es la base de todos los Widgets o controles que componen una Activity.
• Button, EditText, TextView, etc.
• Widget: Deriva de la clase View.
• Permite dibujar: ImageView, FrameLayout, EditText, Button.
• Layout: Es un Widget particular. Actúa como contenedor de otros widgets.
• Define el lugar de la pantalla en el que se dibujarán los widgets, según determinadas reglas.

La clase View



Fuente: *Le Tutoriel de Android UI Layouts* https://o7planning.org/fr/10423/tutoriel-android-ui-layouts

Jerarquía de vistas con parámetros de diseño asociados con cada vista



Fuente: *Diseños* https://developer.android.com/guide/topics/ui/declaring-layout?hl=es

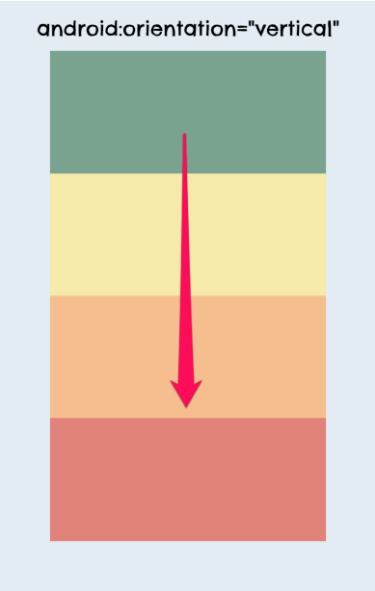
Layouts estandar

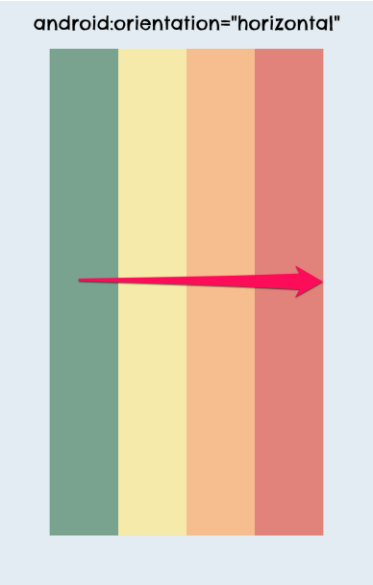


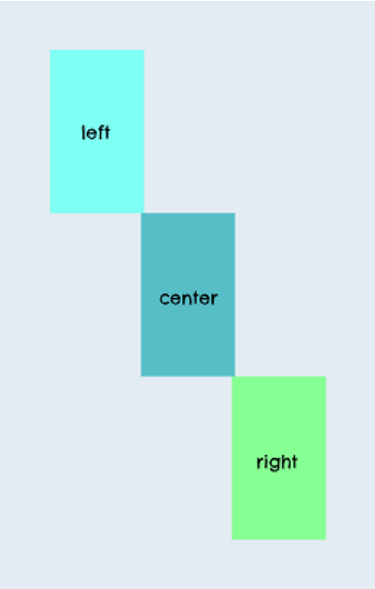
Layout	Descripción
LinearLayout  (Horizontal)  (Vertical)	**LinearLayout** es un grupo de vistas que alinea todos los hijos en una única dirección, ya sea vertical u horizontalmente
RelativeLayout 	**RelativeLayout** es un grupo de vistas que muestra vistas secundarias en posiciones relativas.
TableLayout 	**TableLayout** es una vista que agrupa vistas en filas y columnas.
GridLayout 	**GridLayout** utiliza una cuadrícula de líneas finas para separar su área de dibujo en: filas, columnas y celdas. Soporta tanto la extensión de filas como de columnas, que juntas permiten que un widget ocupe un rango rectangular de celdas ubicadas una al lado de la otra.
FrameLayout 	**FrameLayout** es un marcador de posición en la pantalla que se puede utilizar para mostrar una sola vista.
AbsoluteLayout	**AbsoluteLayout** permite especificar la ubicación exacta de los hijos. Organiza las vistas secundarias (hijos) de acuerdo con las coordenadas x, y.

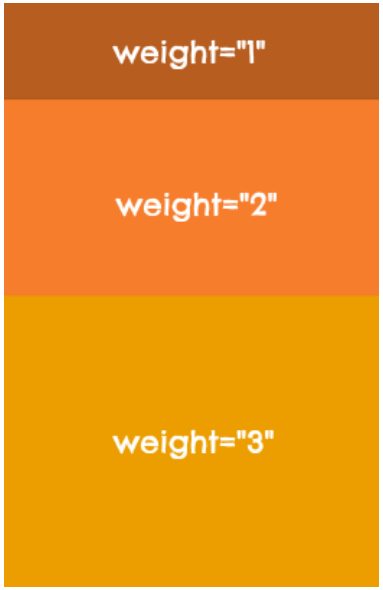
Fuente: *Diseños* https://developer.android.com/guide/topics/ui/ded

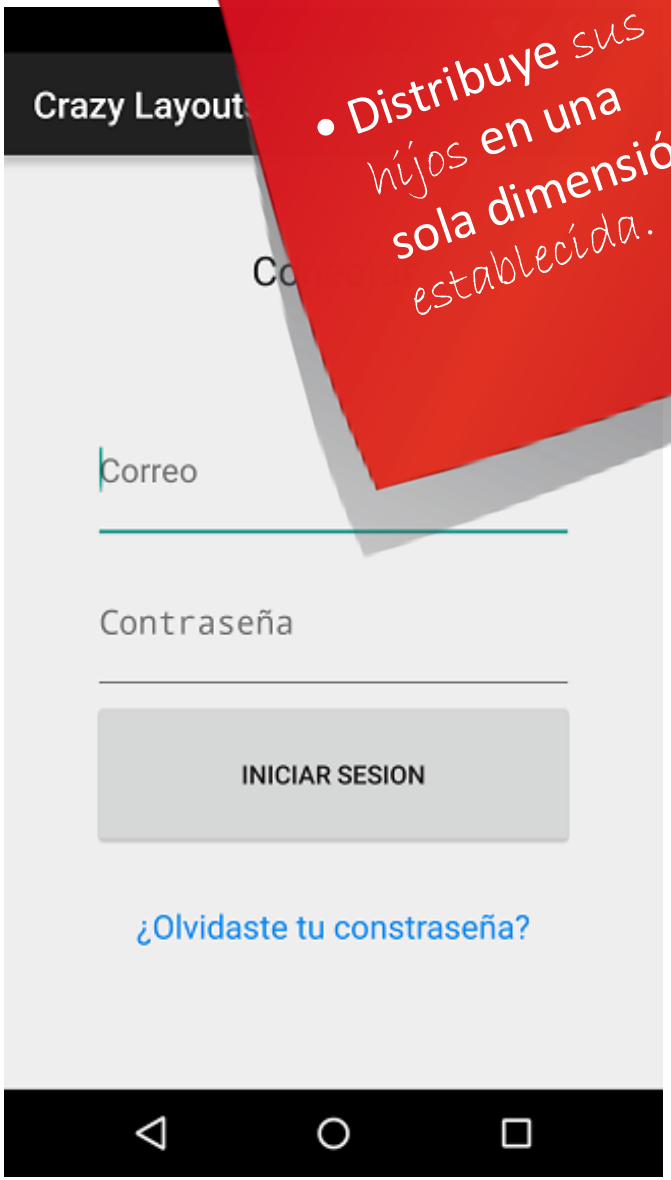
LinearLayout





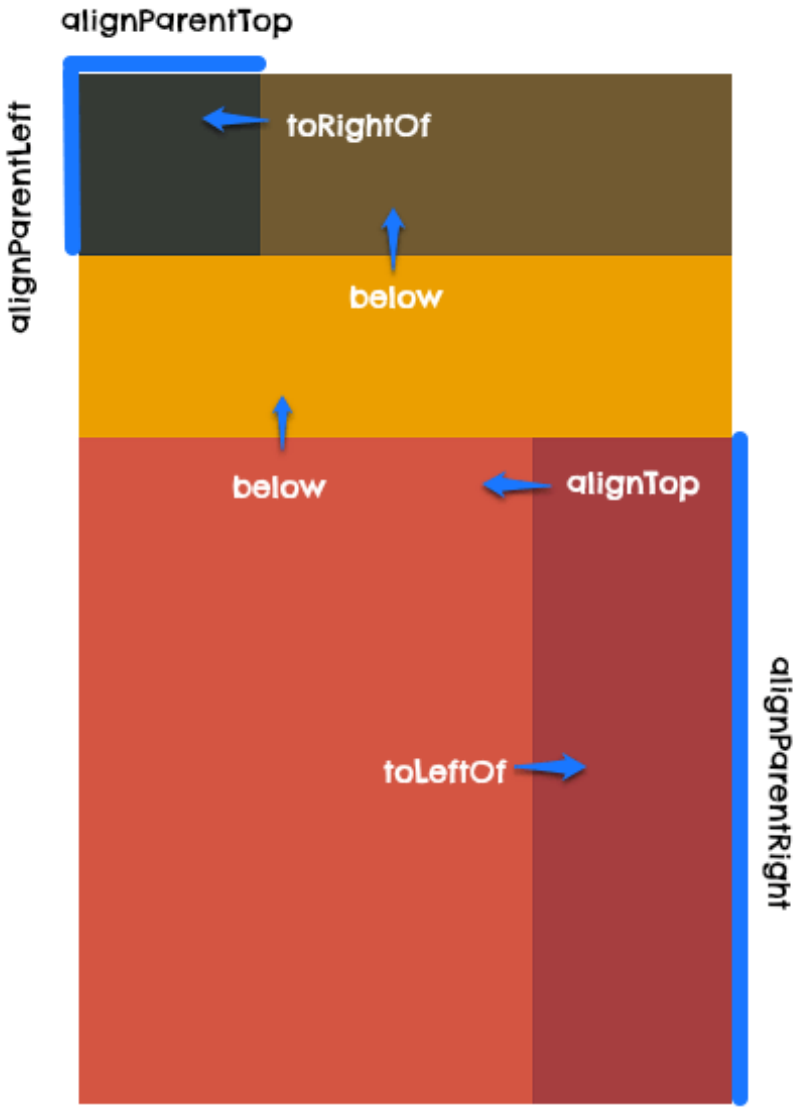


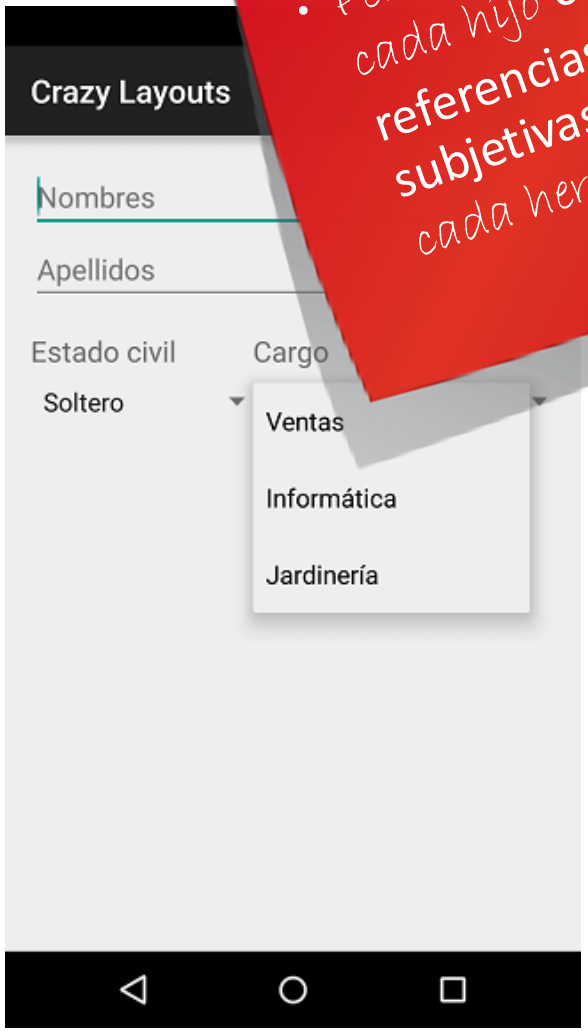




- Distribuye sus hijos en una sola dimensión establecida.

RelativeLayout (1)

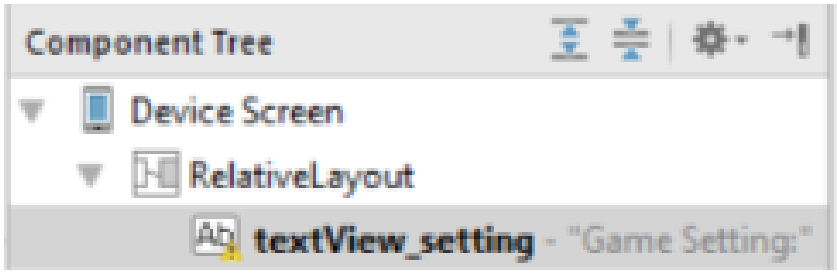




- Permite alinear cada hijo con referencias subjetivas a cada hermano.

RelativeLayout (2)

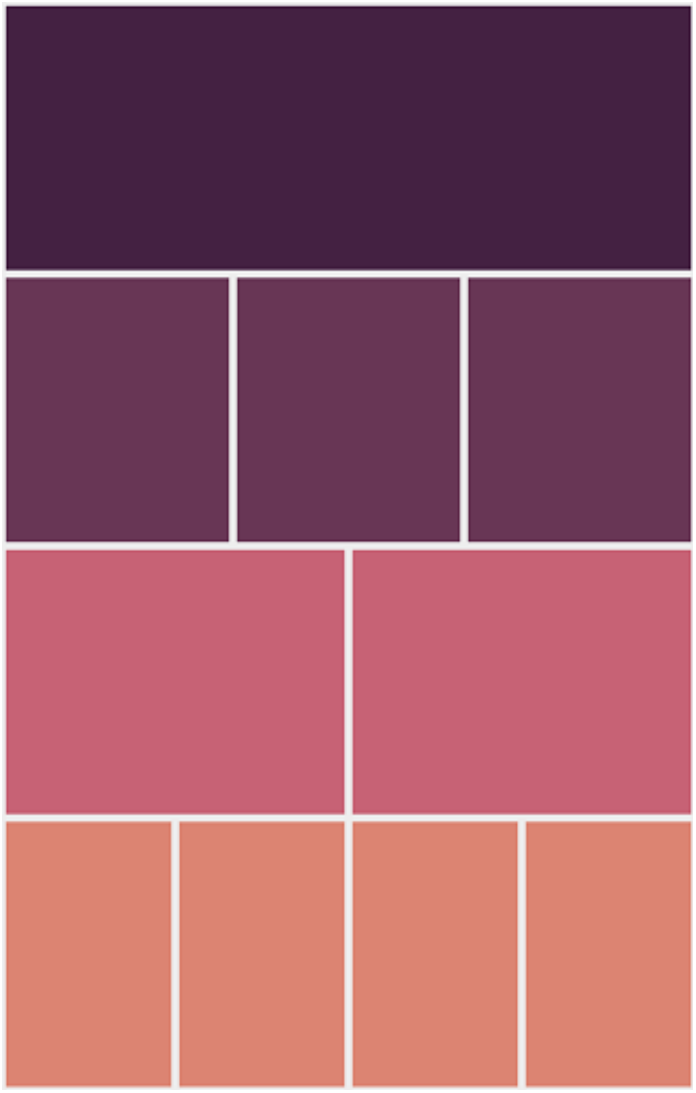


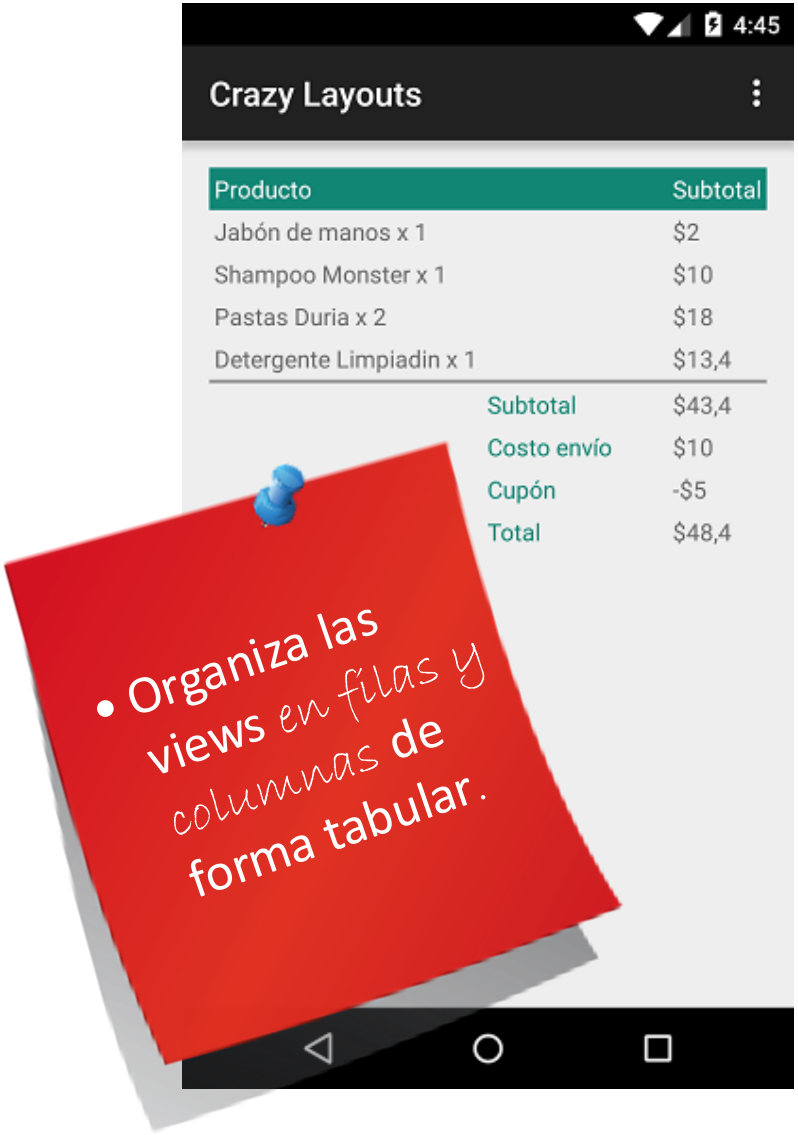


- Si hay muchos **LinearLayout** anidados, se pueden reemplazar con un solo **RelativeLayout**.

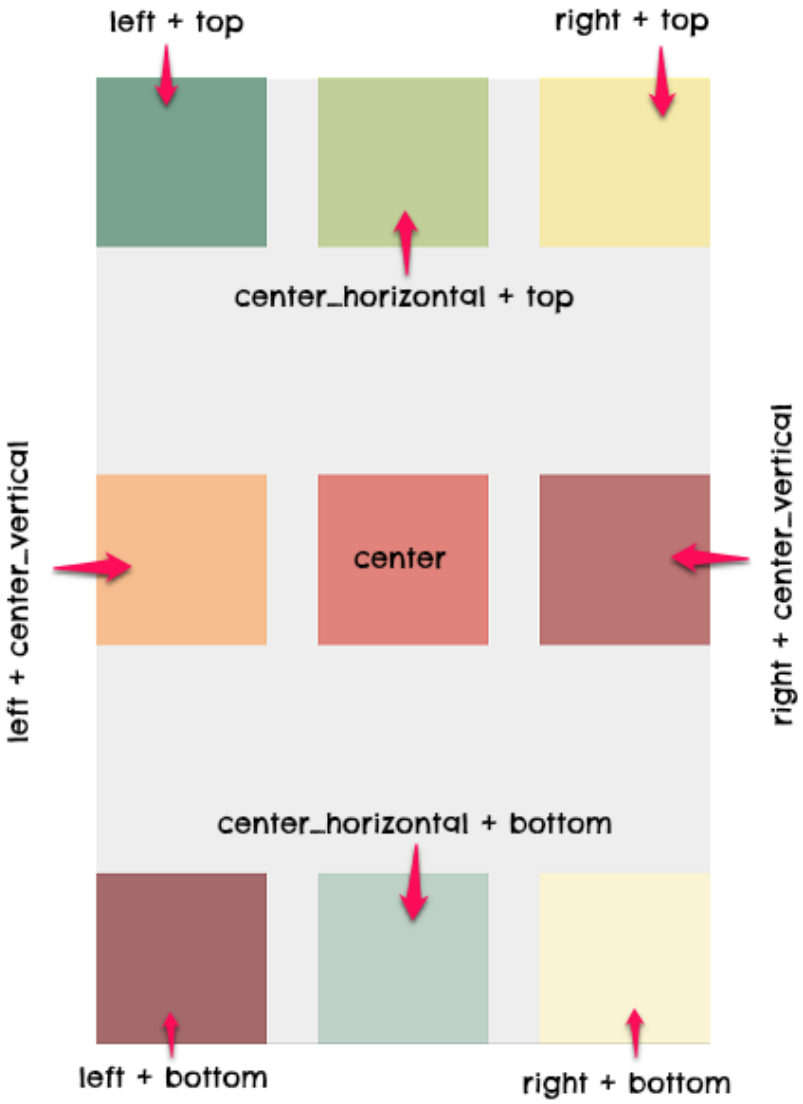
- **Mejora el desempeño** de la aplicación.

TableLayout





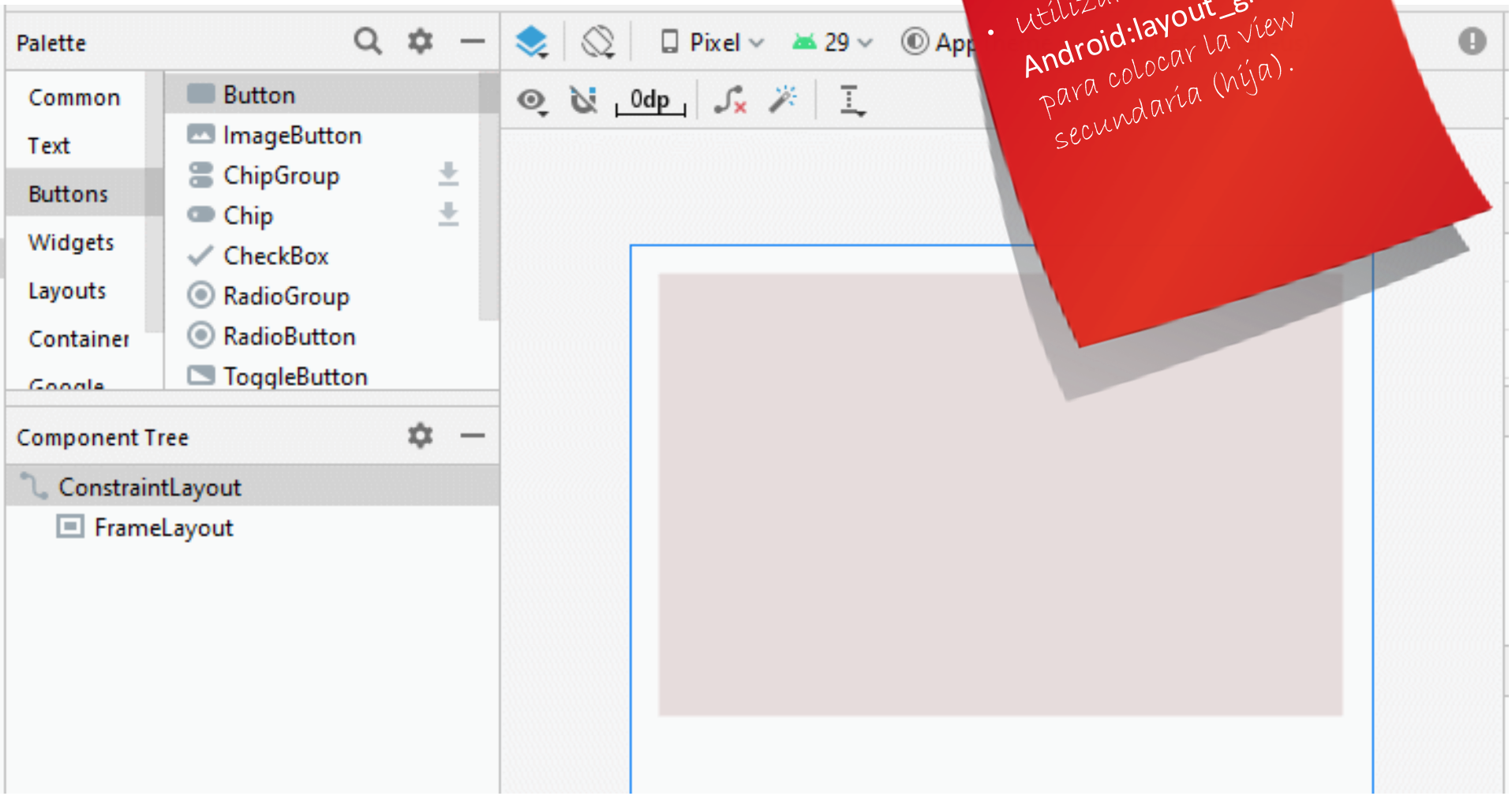
FrameLayout (1)



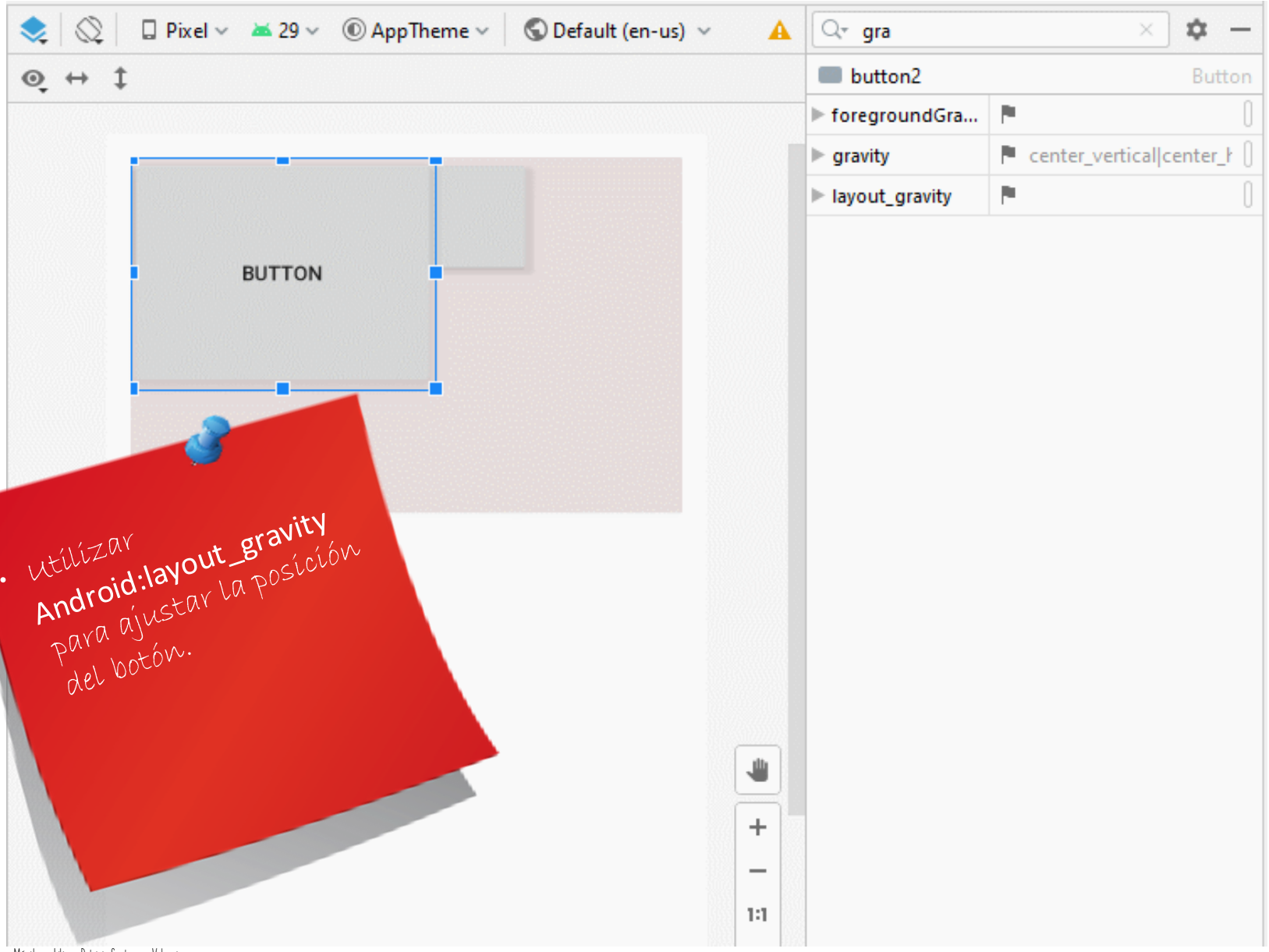
- Muestra un solo elemento en la pantalla.



FrameLayout (2)

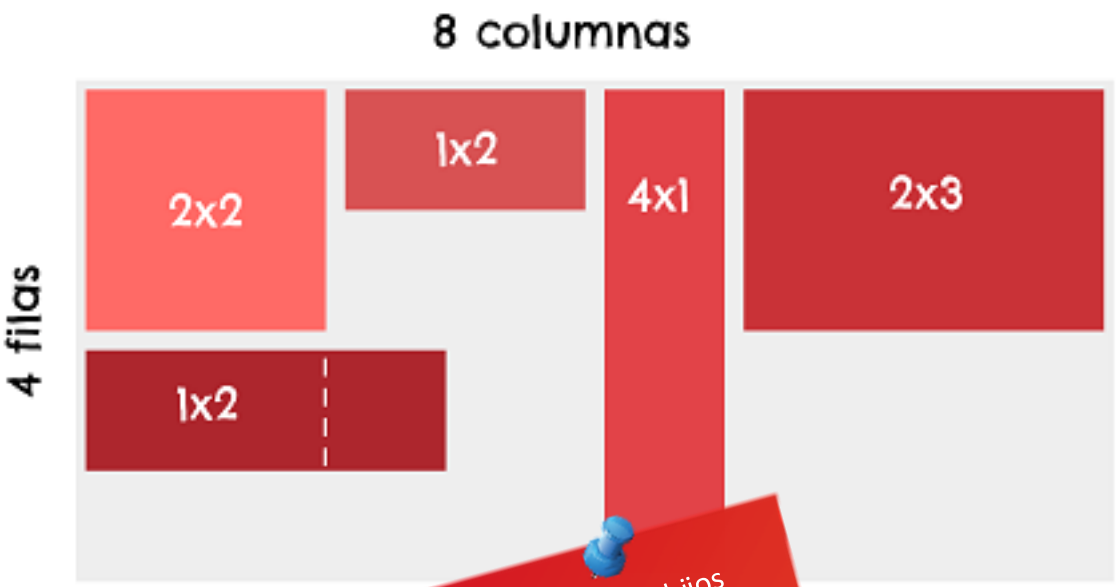


FrameLayout (3)

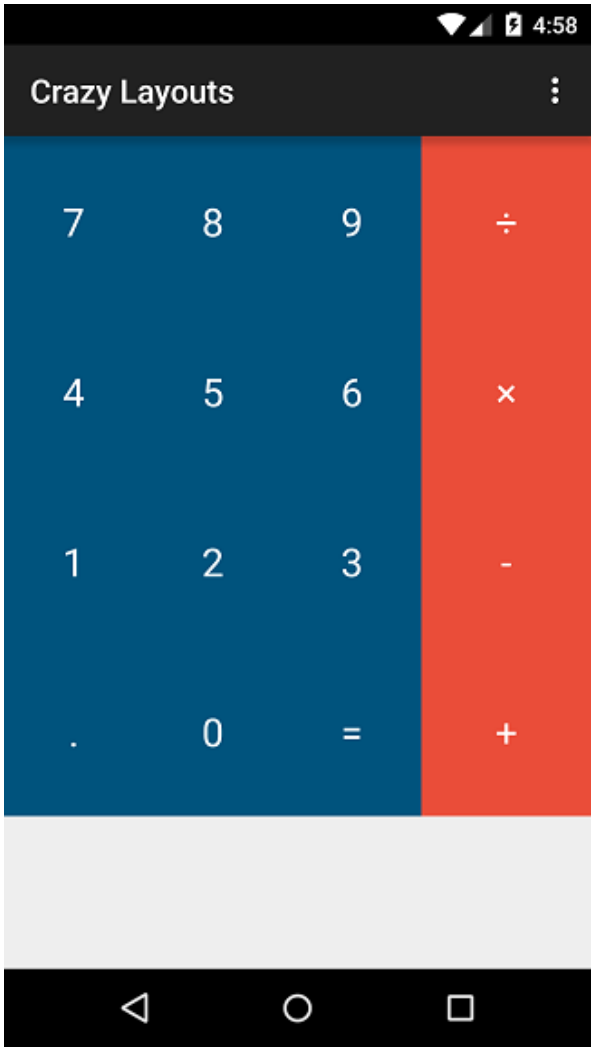


- Utilizar **Android:layout_gravity** para ajustar la posición del botón.

GridLayout (1)



- Alinea sus *elementos* hijos en una cuadrícula (*grilla ó grid*).
- *Evitar anidar linear layouts* para crear diseños complejos.
- *Basado en un* sistema de índices *para las columnas que empieza en cero.*



GridLayout (2)

Márgenes, tamaños, alineamientos y gravedad

- Funcionan igual que en **LinearLayout** y con las mismas constantes: left, top, right, bottom, center_horizontal, fill, etc.

Flexibilidad

- No asocia datos con filas. Todo lo relacionado con la **alineación y la flexibilidad** está **asociado con los propios componentes**.
- La **posición de los subcomponentes** en la cuadrícula se define mediante los atributos **layout_row** y **layout_column**.

Layout_columnWeight

- Si especifica un valor diferente de cero para **layout_columnWeight** es necesario establecer un valor **layout_gravity** para el objeto, esto **es obligatorio**, de lo contrario, el objeto no se puede mostrar en la cuadrícula.

Layout_rowSpan y layout_columnSpan

- Para combinar celdas contínuas en una columna o fila, respectivamente.
- **layout_rowWeight**, el valor predeterminado es 0.



Formatos y unidades de medida

Formatos comunes para Views y ViewGroups

ATRIBUTO	DESCRIPCIÓN
`Layout_width`	Especifica el ancho del `View` o `ViewGroup`
`Layout_heigth`	Especifica el alto del `View` o `ViewGroup`
`Layout_marginTop`	Especifica el espacio extra del lado superior del `View` o `ViewGroup`
`Layout_marginBotton`	Especifica el espacio extra del lado inferior del `View` o `ViewGroup`
`Layout_marginLeft`	Especifica el espacio extra del lado izquierdo del `View` o `ViewGroup`
`Layout_marginRight`	Especifica el espacio extra del lado derecho del `View` o `ViewGroup`
`Layout_gravity`	Especifica cómo se posicionan los hijos de las `Views`
`Layout_weigth`	Especifica la cantidad de espacio extra que se debe asignar a la `View`
`Layout_x`	Especifica la coordenada `X` de la View o `ViewGroup`
`Layout_y`	Especifica la coordenada `Y` de la View o `ViewGroup`

Unidades de medida

UNIDADES	DESCRIPCIÓN
`dp`	(*Density Independent Pixel*) •160 `dp` = una pulgada física de la pantalla. •Cuando se especifica la dimensión de las `Views` en un layout. •Se puede expresar como `dp` o `dpi`.
`sp`	(*Scale Independent pixel*) •Similar a dp. •Para especificar tamaño de las fuentes.
`pt`	(*Point*) •Equivale a 1/72 de una pulgada. •Se basa en el tamaño físico de la pantalla.
`px`	(*Pixel*) •Corresponde a los pixeles actuales en una pantalla. •No recomendable, ya que la IU no se podrá renderizar correctamente en dispositivos con diferentes tamaños de pantalla.

Bibliografía

- Developer Android. (2026, 15 de septiembre). Aspectos básicos de la App. https://developer.android.com/
- Le Tutoriel de Android UI Layouts. (2026, 15 de septiembre). https://devstory.net/10423/android-ui-layouts
- Develou. (2026, 15 de septiembre). Tutorial de Layouts en Android. https://www.develou.com/tutorial-layouts-en-android/
- Pixabay. (2026, 15 de septiembre). Pixabay. https://pixabay.com/es/
- Genially. (2026, 25 de septiembre). Genially. https://genial.ly/education/



TIPOS DE LAYOUTS

Liliana Patricia Santacruz Valencia

liliana.santacruz@urjc.es

Universidad
Rey Juan Carlos